CAPSTONE PROJECT LEVEL 2

Smart Banking Assistant

Course Code: BFSI-ARAG-002 | Difficulty: Advanced | Duration: 32 hours

"Ask anything — about our products or your account."

Refer to the Knowledgebase file here.

https://docs.google.com/document/d/1oiZymgV5TGnSIOIN45keWhoBlUYrv-IWRdEl0PUvK-A/edit?usp=sharing

PROJECT OVERVIEW

1.1 What is This Project?

A Smart Banking Assistant built for the BFSI domain that goes beyond basic retrieval. It combines multimodal document ingestion (PDFs, images, tables, charts), an intelligent LangGraph-based retrieval agent with retry logic and hybrid search, and a direct RDBMS integration to answer structured data queries — all exposed via a FastAPI backend.

1.2 Business Problem

- Relationship managers and analysts deal with mixed-format documents: product brochures (images), loan statements (tables), policy PDFs (text), and scanned forms (image+text)
- Customers ask questions that require both unstructured knowledge (policy docs) and structured transactional data (account history, purchase records)
- A basic RAG system silently returns empty results when the first search phrase fails — no fallback, no retry
- Analysts waste time switching between the document portal and the core banking system for a single query

1.3 Sample Questions the System Must Answer

From Documents (RAG):

- What are the foreclosure charges applicable for a home loan disbursed before 2022?
- Show me the interest rate table from the Fixed Deposit product brochure.
- What does the credit card T&C say about international transaction fees?

From RDBMS (Structured Queries):

- Give me the last 3 months purchase history of customer account 1345367.
- What is the current outstanding balance and next EMI due date for loan account L-789012?
- List all transactions above ₹50,000 for account 1345367 in the last quarter.

1.4 End Goal

User query → LangGraph Agent decides: RAG path or RDBMS path (or both) → RAG path: multimodal retrieval with hybrid search + reranking + retry → RDBMS path: SQL query generation + execution → Merged, grounded response returned via FastAPI

SYSTEM ARCHITECTURE

2.1 Architecture Overview

Multimodal Documents (PDF + Images + Tables) → Document Loader (PyMuPDF / Unstructured) → Multimodal Chunking → Image captioning (vision model) → Text + Image Embeddings → PGVector Storage

Query → LangGraph Agent → Hybrid Search (Vector + FTS) → Reranker → [Retry with 2 alternate phrases if 0 chunks] → Top-K Chunks → LLM Response

2.2 Multimodal Ingestion Strategy

Document Element	Handling Strategy
Text paragraphs	RecursiveCharacterTextSplitter (512 tokens, 100 overlap)
Tables (PDF)	Camelot / pdfplumber → Markdown table → chunked
Images / Charts	Vision LLM caption → text stored as chunk with image_ref
Scanned docs	OCR (pytesseract / Azure OCR) → text pipeline
Mixed pages	Page-level segmentation before element-level extraction

Metadata to preserve per chunk:

Field	Description
id	UUID
content	TEXT — chunk text or image caption
embedding	vector(1536)
document_id	UUID
chunk_type	VARCHAR — text / table / image_caption
source_page	INT
document_name	VARCHAR
product_category	VARCHAR — loan / deposit / card / insurance
created_at	TIMESTAMP

2.3 LangGraph Agent Design

The retrieval agent is implemented as a LangGraph StateGraph with the following nodes:

Node	Responsibility
query_classifier	Decides: RAG / SQL / Hybrid path

rag_retriever	Runs hybrid search (vector + FTS) on PGVector
reranker	Cross-encoder reranking of top-K chunks
query_rewriter	Generates 2 alternate search phrases on zero-result
sql_generator	Converts natural language to parameterized SQL
sql_executor	Runs safe, read-only SQL on Postgres RDBMS
response_generator	Assembles context + prompt → LLM call

Retry Logic:

```
attempt_1: search(original_query)
  → if len(chunks) == 0:
     attempt_2: search(rewritten_phrase_1)
       → if len(chunks) == 0:
          attempt_3: search(rewritten_phrase_2)
            → if len(chunks) == 0: return "No relevant documents found"
```

2.4 Hybrid Retrieval Strategy

- Vector Search: Top-K (k=5) cosine similarity using PGVector
- Full-Text Search (FTS): PostgreSQL tsvector + tsquery for keyword matching
- Hybrid Merge: Reciprocal Rank Fusion (RRF) to combine scores
- Reranking: cross-encoder/ms-marco-MiniLM-L-6-v2 re-scores top-10 → returns top-5

2.5 RDBMS Integration (Postgres)

The system connects to a separate transactional Postgres schema representing core banking data.

Table	Key Fields
accounts	account_id (PK), customer_name, account_type, branch_code, created_at
transactions	txn_id (PK), account_id (FK), txn_date, txn_type, amount, description, channel
loan_accounts	loan_id (PK), account_id (FK), principal, outstanding, emi_amount, next_emi_date, loan_type

SQL Safety Rules (enforce in code):

- Only SELECT statements allowed — no INSERT/UPDATE/DELETE
- Parameterized queries only (no string concatenation)
- Query result rows capped at 100
- Sensitive fields (PAN, mobile) masked in response

2.6 LLM Configuration

Parameter	Value
Model	Google Gemini / OpenAI GPT-4o (configurable)
Temperature	0.1 (near-deterministic)
Max Tokens	2048

System Prompt	Grounded answers, citation of chunk source, no hallucination of numbers
Input Format	Retrieved chunks OR SQL result table + user query

COURSE MODULE MAPPING

- Module 2.1–2.4: LangChain basics, embeddings, vector search, PGVector setup
- Module 3.1: Multimodal document loading — PDF text, table extraction, image captioning
- Module 3.2: PGVector schema with chunk_type metadata; hybrid FTS + vector queries
- Module 3.3: LangGraph agent design — StateGraph, nodes, conditional edges, retry logic
- Module 3.4: RDBMS integration — NL-to-SQL node, Postgres connection, safe query execution
- Module 3.5: FastAPI endpoints, request/response validation, async support
- Module 4.1–4.3: LangSmith tracing across all agent nodes, cost monitoring per query path

TECHNOLOGY STACK

Component	Technology
Language	Python 3.10+
Agentic Framework	LangGraph (StateGraph)
RAG Framework	LangChain v1+
LLM	Google Gemini / OpenAI GPT-4o
Embeddings	OpenAI text-embedding-3-small or Google Embedding Model
Vector Database	PostgreSQL 16+ with pgvector extension
RDBMS	PostgreSQL 16+ (separate schema for transactional data)
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2 (sentence-transformers)
PDF Parsing	PyMuPDF, pdfplumber, Camelot
OCR	pytesseract / Azure Document Intelligence
API Framework	FastAPI 0.100+
Async Support	asyncio, httpx
Monitoring	LangSmith

API SPECIFICATIONS

Endpoint: POST /api/v1/query

Request:

{ "query": "Give me the last 3 months purchase history of customer account 1345367", "session_id": "optional-uuid" }

Response (200 OK):

Field	Description

query	Original user query
answer	Grounded answer text
query_path	"rag" / "sql" / "hybrid" — which path was used
citations	Array of chunk sources (for RAG path)
sql_query	The generated SQL (for SQL path, for auditability)
sql_result	Raw table result (for SQL path)
retry_count	Number of retrieval retries performed (0–2)
confidence_score	Float 0–1
langsmith_trace_id	UUID for debugging

Endpoint: POST /api/v1/ingest

Request:

Multipart file upload (PDF, PNG, JPG)

Response:

Field	Description
document_id	UUID assigned to ingested document
chunks_created	Total chunks stored
chunk_types	Breakdown: {text: N, table: N, image_caption: N}
status	"success" / "partial" / "failed"

EVALUATION CRITERIA & GRADING RUBRIC

Grading Components (100 points)

Component	Weight	Details
Multimodal Ingestion	20%	PDF text + table + image caption extraction, correct metadata tagging
LangGraph Agent	25%	Correct node design, conditional routing (RAG vs SQL), retry logic functional
Hybrid Retrieval + Reranking	20%	Vector + FTS working, RRF merge, reranker applied correctly
RDBMS Integration	15%	NL-to-SQL generation, safe parameterized query execution, correct results
API & Integration	10%	FastAPI endpoints, request/response validation, error handling
LangSmith Monitoring	10%	Tracing across all agent nodes, retry events captured
Bonus: Retry Evaluation	+5%	Demonstrate retry triggering and successful recovery on 3 test queries

LangSmith Monitoring Strategy

- Trace every LangGraph node individually (classifier, retriever, reranker, SQL executor, generator)
- Tag traces with: query_path, retry_count, chunk_type_retrieved, response_time_ms
- SLA Targets: Response time <2000ms (p95), Cost <$0.02/query, Hallucination rate <10%

KEY SUCCESS METRICS

- Retrieval Precision: >80% of top-5 retrieved chunks are relevant
- Retry Recovery Rate: >60% of zero-result queries recover on retry
- SQL Accuracy: >90% of structured queries return correct data
- API Response Latency: <2000ms (p95)
- Cost Per Query: <$0.02
- Zero unsafe SQL (no mutations, no unparameterized queries)

DELIVERABLES

- Complete Python source code (src/ folder with modular components)
- LangGraph agent definition with node implementations and StateGraph config
- FastAPI application with Swagger documentation
- PostgreSQL schema: pgvector table (multimodal chunks) + transactional RDBMS schema with seed data
- Sample BFSI documents for ingestion (product brochures, policy PDFs, rate cards — sourced by trainee)
- LangSmith dashboard screenshot showing traces across all agent nodes
- Postman/curl scripts for API testing (ingest + query endpoints)

APPENDIX: SQL SEED DATA

The following seed data file is provided for use with the NorthStar Bank Smart Banking Assistant project. It creates all necessary tables and populates them with realistic sample data including customer accounts, transaction history, loan accounts, fixed deposits, and credit card records. Run this script against a PostgreSQL 16+ database before starting the project.

Database Tables Overview

Table	Rows	Description
accounts	8	Customer bank accounts (savings, salary, current)
transactions	44	Debit/credit transaction history with merchant and category data
loan_accounts	6	Home loans, personal loans, auto loans with EMI details
fixed_deposits	6	FD records with tenure, interest rate, maturity details
credit_cards	4	Card variants with credit limit and outstanding amounts
card_transactions	12	Credit card purchases including international transactions

Key Sample Accounts

Account ID	Customer Name	Type	Branch
1345367	James Mitchell	Savings	CHN001
2456789	Sarah Thompson	Salary	MUM002
3567890	Robert Clarke	Current	DEL003
4678901	Emily Watson	Savings	HYD004
5789012	Daniel Foster	Salary	BLR005
6890123	Laura Bennett	Savings	CHN001
7901234	Michael Harrington	Current	DEL003
8012345	Catherine Ellis	Savings	BLR005

Sample Queries for Testing the NL-to-SQL Node

Q1 — Transaction History

Natural Language: Give me the last 3 months purchase history of customer account 1345367

SQL:

```
SELECT txn_date, txn_type, amount, description, merchant_name, category,
channel
FROM transactions
```

```
WHERE account_id = '1345367'
  AND txn_date >= CURRENT_DATE - INTERVAL '3 months'
ORDER BY txn_date DESC;
```

Q2 — Loan Details

Natural Language: What is the outstanding balance and next EMI date for loan L-789012?

SQL:

```
SELECT loan_id, loan_type, outstanding, emi_amount, next_emi_date,
interest_rate
FROM loan_accounts
WHERE loan_id = 'L-789012';
```

Q3 — High-Value Transactions

Natural Language: List all transactions above $50,000 for account 1345367

SQL:

```
SELECT txn_date, txn_type, amount, description, channel
FROM transactions
WHERE account_id = '1345367' AND amount > 50000
ORDER BY txn_date DESC;
```

Q4 — Spend by Category

Natural Language: What did account 1345367 spend the most on last quarter?

SQL:

```
SELECT category, SUM(amount) AS total_spent, COUNT(*) AS txn_count
FROM transactions
WHERE account_id = '1345367'
  AND txn_type = 'debit'
  AND txn_date >= DATE_TRUNC('quarter', CURRENT_DATE) - INTERVAL '3 months'
GROUP BY category ORDER BY total_spent DESC;
```

Q5 — Active Fixed Deposits

Natural Language: Show me all active FDs for account 1345367

SQL:

```
SELECT fd_id, principal, interest_rate, maturity_date, maturity_amount,
interest_payout
FROM fixed_deposits
WHERE account_id = '1345367' AND status = 'active';
```

Q6 — International Card Transactions

Natural Language: Show international transactions on credit card CC-881001

SQL:

```
SELECT txn_date, amount, merchant_name, currency, category
FROM card_transactions
WHERE card_id = 'CC-881001' AND is_international = TRUE
ORDER BY txn_date DESC;
```

Complete seed_data.sql

The full SQL script follows. It includes CREATE TABLE statements, all INSERT data, and commented sample queries for each test scenario.

```
-- ============================================================
-- NorthStar Bank — Seed Data for Smart Banking Assistant
-- Capstone Project Level 2 (BFSI-ARAG-002)
-- PostgreSQL 16+
-- ============================================================

-- ─────────────────────────────────────────────────────────────
-- 0. EXTENSIONS
-- ─────────────────────────────────────────────────────────────
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS vector;

-- ─────────────────────────────────────────────────────────────
-- 1. SCHEMA
-- ─────────────────────────────────────────────────────────────

-- 1.1 Accounts
CREATE TABLE IF NOT EXISTS accounts (
    account_id      VARCHAR(20)  PRIMARY KEY,
    customer_name   VARCHAR(100) NOT NULL,
    account_type    VARCHAR(20)  NOT NULL CHECK (account_type IN
('savings','current','salary')),
    branch_code     VARCHAR(10)  NOT NULL,
    ifsc_code       VARCHAR(15),
    mobile          VARCHAR(15),   -- store masked: e.g. XXXXXXX890
    email           VARCHAR(100),
    kyc_status      VARCHAR(20)  DEFAULT 'verified',
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- 1.2 Transaction History
CREATE TABLE IF NOT EXISTS transactions (
    txn_id          UUID         PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id      VARCHAR(20)  REFERENCES accounts(account_id),
    txn_date        DATE         NOT NULL,
    txn_type        VARCHAR(10)  NOT NULL CHECK (txn_type IN ('debit','credit')),
    amount          NUMERIC(15,2) NOT NULL,
    balance_after   NUMERIC(15,2),
    description     VARCHAR(200),
    channel         VARCHAR(20)  CHECK (channel IN
('ATM','UPI','NEFT','RTGS','IMPS','branch','online','POS')),
    merchant_name   VARCHAR(100),
    category        VARCHAR(50),
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- 1.3 Loan Accounts
CREATE TABLE IF NOT EXISTS loan_accounts (
    loan_id         VARCHAR(20)  PRIMARY KEY,
    account_id      VARCHAR(20)  REFERENCES accounts(account_id),
```

```
    loan_type       VARCHAR(30)  NOT NULL CHECK (loan_type IN
('home_loan','personal_loan','auto_loan','gold_loan')),
    principal       NUMERIC(15,2) NOT NULL,
    outstanding     NUMERIC(15,2) NOT NULL,
    disbursed_date  DATE,
    emi_amount      NUMERIC(15,2),
    next_emi_date   DATE,
    interest_rate   NUMERIC(5,2),
    tenure_months   INT,
    emi_paid        INT DEFAULT 0,
    status          VARCHAR(20)  DEFAULT 'active',
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- 1.4 Fixed Deposits
CREATE TABLE IF NOT EXISTS fixed_deposits (
    fd_id           VARCHAR(20)  PRIMARY KEY,
    account_id      VARCHAR(20)  REFERENCES accounts(account_id),
    principal       NUMERIC(15,2) NOT NULL,
    interest_rate   NUMERIC(5,2) NOT NULL,
    tenure_days     INT          NOT NULL,
    start_date      DATE         NOT NULL,
    maturity_date   DATE         NOT NULL,
    maturity_amount NUMERIC(15,2),
    interest_payout VARCHAR(20)  DEFAULT 'at_maturity',
    status          VARCHAR(20)  DEFAULT 'active',
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- 1.5 Credit Cards
CREATE TABLE IF NOT EXISTS credit_cards (
    card_id         VARCHAR(20)  PRIMARY KEY,
    account_id      VARCHAR(20)  REFERENCES accounts(account_id),
    card_variant    VARCHAR(30),
    credit_limit    NUMERIC(15,2),
    available_limit NUMERIC(15,2),
    outstanding_amt NUMERIC(15,2) DEFAULT 0,
    due_date        DATE,
    min_due         NUMERIC(15,2) DEFAULT 0,
    status          VARCHAR(20)  DEFAULT 'active',
    issued_date     DATE,
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- 1.6 Credit Card Transactions
CREATE TABLE IF NOT EXISTS card_transactions (
    txn_id          UUID         PRIMARY KEY DEFAULT uuid_generate_v4(),
    card_id         VARCHAR(20)  REFERENCES credit_cards(card_id),
    txn_date        DATE         NOT NULL,
    txn_type        VARCHAR(20)  CHECK (txn_type IN
('purchase','cashadvance','payment','refund','fee')),
    amount          NUMERIC(15,2) NOT NULL,
    merchant_name   VARCHAR(100),
    category        VARCHAR(50),
    is_international BOOLEAN     DEFAULT FALSE,
    currency        VARCHAR(5)   DEFAULT 'INR',
    created_at      TIMESTAMP    DEFAULT NOW()
);

-- ═══════════════════════════════════════════════════════════════
-- 2. SEED DATA — ACCOUNTS
-- ═══════════════════════════════════════════════════════════════
INSERT INTO accounts (account_id, customer_name, account_type, branch_code, ifsc_code,
mobile, email, kyc_status, created_at) VALUES
('1345367',  'James Mitchell',          'savings',  'CHN001', 'NSBK0CHN001',
'XXXXXXX890', 'james.mitchell@email.com',     'verified',  '2019-03-15 10:00:00'),
('2456789',  'Sarah Thompson',          'salary',   'MUM002', 'NSBK0MUM002',
'XXXXXXX123', 'sarah.thompson@email.com',     'verified',  '2020-07-22 11:30:00'),
('3567890',  'Robert Clarke',           'current',  'DEL003', 'NSBK0DEL003',
'XXXXXXX456', 'robert.clarke@bizmail.com',    'verified',  '2018-11-05 09:15:00'),
('4678901',  'Emily Watson',          'savings',  'HYD004', 'NSBK0HYD004', 'XXXXXXX789',
```

```
'emily.watson@email.com',   'verified',  '2021-02-18 14:45:00'),
('5789012',  'Daniel Foster',         'salary',   'BLR005', 'NSBK0BLR005',
'XXXXXXX321', 'daniel.foster@email.com',   'verified',  '2022-06-01 08:00:00'),
('6890123',  'Laura Bennett',         'savings',  'CHN001', 'NSBK0CHN001',
'XXXXXXX654', 'laura.bennett@email.com',   'verified',  '2020-09-30 16:20:00'),
('7901234',  'Michael Harrington',         'current',  'DEL003', 'NSBK0DEL003',
'XXXXXXX987', 'michael.harrington@corp.com',    'verified',  '2017-04-10 11:00:00'),
('8012345',  'Catherine Ellis',  'savings',  'BLR005', 'NSBK0BLR005', 'XXXXXXX147',
'catherine.ellis@email.com',        'verified',  '2023-01-14 09:30:00');

-- ─────────────────────────────────────────────────────────────
-- 3. SEED DATA — TRANSACTIONS (last 6 months for account 1345367)
-- ─────────────────────────────────────────────────────────────

-- Account 1345367 — James Mitchell (primary sample account)
INSERT INTO transactions (account_id, txn_date, txn_type, amount, balance_after,
description, channel, merchant_name, category) VALUES
-- January 2026
('1345367', '2026-01-03', 'credit', 85000.00, 185000.00, 'Salary Credit - January
2026',      'NEFT',   'Apex Solutions Inc',    'salary'),
('1345367', '2026-01-05', 'debit',  15000.00, 170000.00, 'Home Loan EMI - January',
'NEFT',   'NorthStar Bank',       'loan_emi'),
('1345367', '2026-01-07', 'debit',   4500.00, 165500.00, 'BigBasket Online Grocery',
'UPI',    'BigBasket',            'groceries'),
('1345367', '2026-01-10', 'debit',   2200.00, 163300.00, 'Swiggy Food Order',
'UPI',    'Swiggy',               'food_dining'),
('1345367', '2026-01-12', 'debit',  12000.00, 151300.00, 'Amazon Shopping',
'online', 'Amazon India',         'shopping'),
('1345367', '2026-01-15', 'debit',   3500.00, 147800.00, 'BESCOM Electricity Bill',
'online', 'BESCOM',               'utilities'),
('1345367', '2026-01-18', 'debit',   8000.00, 139800.00, 'Tata Motors Service Center',
'POS',    'Tata Service',         'automobile'),
('1345367', '2026-01-20', 'credit',  5000.00, 144800.00, 'UPI Transfer Received - Kevin
Walsh',   'UPI',    NULL,                   'transfer'),
('1345367', '2026-01-22', 'debit',   1800.00, 143000.00, 'Netflix Subscription',
'online', 'Netflix',              'entertainment'),
('1345367', '2026-01-25', 'debit',  25000.00, 118000.00, 'HDFC Life Insurance Premium',
'NEFT',   'HDFC Life',            'insurance'),
('1345367', '2026-01-28', 'debit',   6700.00, 111300.00, 'Apollo Pharmacy',
'UPI',    'Apollo Pharmacy',      'medical'),
('1345367', '2026-01-31', 'debit',   3200.00, 108100.00, 'Airtel Postpaid Bill',
'online', 'Airtel',               'utilities'),

-- February 2026
('1345367', '2026-02-03', 'credit', 85000.00, 193100.00, 'Salary Credit - February
2026',     'NEFT',   'Apex Solutions Inc',    'salary'),
('1345367', '2026-02-05', 'debit',  15000.00, 178100.00, 'Home Loan EMI - February',
'NEFT',   'NorthStar Bank',       'loan_emi'),
('1345367', '2026-02-08', 'debit',   5200.00, 172900.00, 'Reliance Smart Grocery',
'POS',    'Reliance Smart',       'groceries'),
('1345367', '2026-02-10', 'debit',  18500.00, 154400.00, 'Croma Electronics -
Headphones',     'POS',    'Croma',                'electronics'),
('1345367', '2026-02-14', 'debit',   3800.00, 150600.00, 'Zomato Valentine Dinner',
'UPI',    'Zomato',               'food_dining'),
('1345367', '2026-02-15', 'debit',   2800.00, 147800.00, 'BWSSB Water Bill',
'online', 'BWSSB',                'utilities'),
('1345367', '2026-02-18', 'debit',  55000.00,  92800.00, 'NEFT to James FD Account',
'NEFT',   NULL,                   'transfer'),
('1345367', '2026-02-20', 'debit',   4100.00,  88700.00, 'Ola Cabs Monthly Pass',
'UPI',    'Ola',                  'transport'),
('1345367', '2026-02-22', 'debit',   9800.00,  78900.00, 'Decathlon Sports Equipment',
'POS',    'Decathlon',            'shopping'),
('1345367', '2026-02-25', 'debit',   1200.00,  77700.00, 'Hotstar Annual Subscription',
'online', 'Disney+ Hotstar',      'entertainment'),
('1345367', '2026-02-28', 'debit',   7500.00,  70200.00, 'Dr. Rajan Clinic
Consultation',      'UPI',    'Apollo Clinic',        'medical'),

-- March 2026
('1345367', '2026-03-03', 'credit', 85000.00, 155200.00, 'Salary Credit - March 2026',
'NEFT',   'Apex Solutions Inc',    'salary'),
('1345367', '2026-03-05', 'debit',  15000.00, 140200.00, 'Home Loan EMI - March',
```

```
'NEFT',   'NorthStar Bank',       'loan_emi'),
('1345367', '2026-03-07', 'debit',   6300.00, 133900.00, 'More Supermarket',
'POS',    'More Supermarket',     'groceries'),
('1345367', '2026-03-10', 'debit',  32000.00, 101900.00, 'Flight Booking - IndiGo',
'online', 'IndiGo Airlines',      'travel'),
('1345367', '2026-03-12', 'debit',  15000.00,  86900.00, 'MakeMyTrip Hotel Booking',
'online', 'MakeMyTrip',           'travel'),
('1345367', '2026-03-15', 'debit',   3500.00,  83400.00, 'BESCOM Electricity Bill',
'online', 'BESCOM',               'utilities'),
('1345367', '2026-03-17', 'debit',   2500.00,  80900.00, 'Swiggy Food Delivery',
'UPI',    'Swiggy',               'food_dining'),
('1345367', '2026-03-19', 'debit',  75000.00,   5900.00, 'NEFT - Advance Tax Q4',
'NEFT',   'Income Tax Dept',      'tax'),
('1345367', '2026-03-20', 'credit', 75000.00,  80900.00, 'UPI Transfer Received -
Bonus',      'UPI',    NULL,                   'transfer'),
('1345367', '2026-03-22', 'debit',   4800.00,  76100.00, 'Nykaa Shopping',
'online', 'Nykaa',                'shopping'),
('1345367', '2026-03-25', 'debit',  12000.00,  64100.00, 'LIC Premium Payment',
'online', 'LIC India',            'insurance'),
('1345367', '2026-03-28', 'debit',   3300.00,  60800.00, 'Airtel Postpaid Bill',
'online', 'Airtel',               'utilities'),
('1345367', '2026-03-31', 'debit',   8500.00,  52300.00, 'D-Mart Shopping',
'POS',    'D-Mart',               'groceries'),

-- Last 2 weeks April 2026
('1345367', '2026-04-01', 'credit', 85000.00, 137300.00, 'Salary Credit - April 2026',
'NEFT',   'Apex Solutions Inc',    'salary'),
('1345367', '2026-04-05', 'debit',  15000.00, 122300.00, 'Home Loan EMI - April',
'NEFT',   'NorthStar Bank',       'loan_emi'),
('1345367', '2026-04-07', 'debit',   5500.00, 116800.00, 'BigBasket Online Grocery',
'UPI',    'BigBasket',            'groceries'),
('1345367', '2026-04-10', 'debit',   3100.00, 113700.00, 'Zomato Order',
'UPI',    'Zomato',               'food_dining');

-- Account 2456789 — Sarah Thompson (sample transactions)
INSERT INTO transactions (account_id, txn_date, txn_type, amount, balance_after,
description, channel, merchant_name, category) VALUES
('2456789', '2026-03-01', 'credit', 120000.00, 250000.00, 'Salary Credit March 2026',
'NEFT',   'GlobalTech Corp',          'salary'),
('2456789', '2026-03-05', 'debit',   22000.00, 228000.00, 'Home Loan EMI',
'NEFT',   'NorthStar Bank',       'loan_emi'),
('2456789', '2026-03-10', 'debit',    8500.00, 219500.00, 'Amazon Shopping',
'online', 'Amazon India',         'shopping'),
('2456789', '2026-03-15', 'debit',    4200.00, 215300.00, 'Grocery - Spencer\'s',
'POS',    'Spencer\'s Retail',    'groceries'),
('2456789', '2026-04-01', 'credit', 120000.00, 335300.00, 'Salary Credit April 2026',
'NEFT',   'GlobalTech Corp',          'salary'),
('2456789', '2026-04-05', 'debit',   22000.00, 313300.00, 'Home Loan EMI',
'NEFT',   'NorthStar Bank',       'loan_emi');

-- ═══════════════════════════════════════════════════════════════
-- 4. SEED DATA — LOAN ACCOUNTS
-- ═══════════════════════════════════════════════════════════════
INSERT INTO loan_accounts (loan_id, account_id, loan_type, principal, outstanding,
disbursed_date, emi_amount, next_emi_date, interest_rate, tenure_months, emi_paid,
status) VALUES
('L-789012', '1345367', 'home_loan',     4500000.00, 3820000.00, '2021-06-15',
15000.00, '2026-05-05', 8.75, 300, 58, 'active'),
('L-892345', '2456789', 'home_loan',     6000000.00, 5400000.00, '2023-01-20',
22000.00, '2026-05-05', 8.50, 360, 39, 'active'),
('L-345678', '4678901', 'personal_loan',  500000.00,  280000.00, '2024-03-10',
9800.00, '2026-05-10', 13.50, 60, 25, 'active'),
('L-456789', '5789012', 'auto_loan',     1200000.00,  950000.00, '2023-09-01',
24500.00, '2026-05-01', 9.25, 60, 19, 'active'),
('L-567890', '7901234', 'home_loan',     8000000.00, 7200000.00, '2022-11-05',
35000.00, '2026-05-05', 9.00, 360, 41, 'active'),
('L-678901', '1345367', 'personal_loan',  300000.00,       0.00, '2020-01-15',
6500.00, NULL,         12.50, 48, 48, 'closed');

-- ═══════════════════════════════════════════════════════════════
-- 5. SEED DATA — FIXED DEPOSITS
```

```
-- ─────────────────────────────────────────────────────────────
INSERT INTO fixed_deposits (fd_id, account_id, principal, interest_rate, tenure_days,
start_date, maturity_date, maturity_amount, interest_payout, status) VALUES
('FD-111001', '1345367', 200000.00, 7.25, 730, '2025-02-18', '2027-02-18', 232900.00,
'at_maturity', 'active'),
('FD-111002', '1345367',  50000.00, 7.10, 365, '2026-01-01', '2027-01-01',  53550.00,
'quarterly',   'active'),
('FD-222001', '2456789', 500000.00, 7.50, 444, '2025-11-01', '2027-01-19', 546250.00,
'at_maturity', 'active'),
('FD-333001', '3567890', 100000.00, 6.75, 548, '2024-06-01', '2025-12-01', 110125.00,
'quarterly',   'matured'),
('FD-444001', '4678901', 150000.00, 7.25, 730, '2025-09-10', '2027-09-10', 172350.00,
'at_maturity', 'active'),
('FD-555001', '6890123',  75000.00, 7.00, 365, '2026-03-01', '2027-03-01',  80250.00,
'at_maturity', 'active');

-- ─────────────────────────────────────────────────────────────
-- 6. SEED DATA — CREDIT CARDS
-- ─────────────────────────────────────────────────────────────
INSERT INTO credit_cards (card_id, account_id, card_variant, credit_limit,
available_limit, outstanding_amt, due_date, min_due, status, issued_date) VALUES
('CC-881001', '1345367', 'NorthStar Gold',      200000.00, 145000.00,  55000.00,
'2026-04-25', 2750.00, 'active', '2021-07-01'),
('CC-882001', '2456789', 'NorthStar Platinum',  500000.00, 420000.00,  80000.00,
'2026-04-28', 4000.00, 'active', '2022-03-15'),
('CC-883001', '5789012', 'NorthStar Classic',    75000.00,  60000.00,  15000.00,
'2026-04-20',  750.00, 'active', '2023-01-10'),
('CC-884001', '8012345', 'NorthStar Signature', 1000000.00,850000.00, 150000.00,
'2026-04-30', 7500.00, 'active', '2024-02-01');

-- ─────────────────────────────────────────────────────────────
-- 7. SEED DATA — CREDIT CARD TRANSACTIONS
-- ─────────────────────────────────────────────────────────────
INSERT INTO card_transactions (card_id, txn_date, txn_type, amount, merchant_name,
category, is_international, currency) VALUES
('CC-881001', '2026-03-02', 'purchase',    3200.00, 'Barbeque Nation',
'food_dining',   FALSE, 'INR'),
('CC-881001', '2026-03-05', 'purchase',   12000.00, 'Myntra',
'shopping',      FALSE, 'INR'),
('CC-881001', '2026-03-10', 'purchase',    8500.00, 'Marriott Hotels',        'travel',
FALSE, 'INR'),
('CC-881001', '2026-03-14', 'purchase',   28500.00, 'Singapore Airlines',     'travel',
TRUE,  'SGD'),
('CC-881001', '2026-03-15', 'purchase',    4200.00, 'Amazon UK',
'shopping',      TRUE,  'GBP'),
('CC-881001', '2026-03-18', 'purchase',    1500.00, 'Spotify',
'entertainment', FALSE, 'INR'),
('CC-881001', '2026-03-22', 'purchase',    9800.00, 'Tanishq Jewellery',
'jewellery',     FALSE, 'INR'),
('CC-881001', '2026-03-25', 'fee',          340.00, 'NorthStar Bank',
'bank_fee',      FALSE, 'INR'),  -- forex markup fee
('CC-881001', '2026-04-01', 'payment',    30000.00, 'NorthStar Payment',
'payment',       FALSE, 'INR'),
('CC-881001', '2026-04-03', 'purchase',    6700.00, 'Reliance Digital',
'electronics',   FALSE, 'INR'),
('CC-881001', '2026-04-07', 'purchase',    2100.00, 'Domino\'s Pizza',
'food_dining',   FALSE, 'INR'),
('CC-881001', '2026-04-10', 'purchase',   18000.00, 'IRCTC Tatkal Ticket',    'travel',
FALSE, 'INR');

-- ─────────────────────────────────────────────────────────────
-- 8. USEFUL SAMPLE QUERIES (as comments — for trainees)
-- ─────────────────────────────────────────────────────────────

-- Q1: Last 3 months purchase history for account 1345367 (James Mitchell)
-- SELECT txn_date, txn_type, amount, description, merchant_name, category, channel
-- FROM transactions
-- WHERE account_id = '1345367'
--   AND txn_date >= CURRENT_DATE - INTERVAL '3 months'
-- ORDER BY txn_date DESC;
```

```
-- Q2: Outstanding balance and next EMI date for loan L-789012
-- SELECT loan_id, loan_type, outstanding, emi_amount, next_emi_date, interest_rate
-- FROM loan_accounts
-- WHERE loan_id = 'L-789012';

-- Q3: All transactions above ₹50,000 for account 1345367
-- SELECT txn_date, txn_type, amount, description, channel
-- FROM transactions
-- WHERE account_id = '1345367' AND amount > 50000
-- ORDER BY txn_date DESC;

-- Q4: Total spend by category last quarter
-- SELECT category, SUM(amount) AS total_spent, COUNT(*) AS txn_count
-- FROM transactions
-- WHERE account_id = '1345367'
--   AND txn_type = 'debit'
--   AND txn_date >= DATE_TRUNC('quarter', CURRENT_DATE) - INTERVAL '3 months'
-- GROUP BY category ORDER BY total_spent DESC;

-- Q5: Active FDs for a customer
-- SELECT fd_id, principal, interest_rate, maturity_date, maturity_amount,
interest_payout
-- FROM fixed_deposits
-- WHERE account_id = '1345367' AND status = 'active';

-- Q6: International transactions on credit card CC-881001
-- SELECT txn_date, amount, merchant_name, currency, category
-- FROM card_transactions
-- WHERE card_id = 'CC-881001' AND is_international = TRUE
-- ORDER BY txn_date DESC;
```